

과제 1은 day2에서 구현한 queue를 연산자 중복 (operator)를 이용해서 queue와 queue를 이어붙이거나 기존 queue에 data를 추가하는 것을 구현하는 것이다.

우선 나는 +로 queue와 queue를 합치고, queue에 data를 추가하는 것을 구현하려고 했다. 그래서 queue.hpp에 `queue<T> operator+(queue<T>& other);` 와 `void operator+(T val);` 를 추가했다. Queue와 queue를 합칠때, queue와 data를 합칠 때 모두 +를 사용하지만 하나는 queue, 다른 하나는 data값을 queue에 추가해야되기 때문에 함수를 오버로딩해서 구현했다.

첫번째 operator+에서는 queue와 queue를 이어 붙이고 result를 반환한다. 하지만 이 result는 어느곳에도 저장되지 않고 그냥 사라진다. 그래서 이 결과 값을 다른 곳에 저장해줘야한다. 그래서 처음에 그냥 대입연산자로 result값을 저장하려고 했다. 하지만 `r = (result)`가 끝나자마자 다 쓴 임시 객체인 result가 해체 되면서 나중에 r 값을 불러오려고 할 때 오류가 발생한다. 이를 해결하기 위해 operator+=을 새로 선언해주었다. operator+=은 새로운 배열을 할당하고 other의 값을 하나씩 복사해서 저장한다. Result가 소멸되는 시점은 `r = p + q` 문장 다음이기 때문에 p + q의 결과인 result가 소멸되기 전에 값을 r에 저장하는 연산자 +=을 새로 정의해준 것이다.

그리고 `r = r`인 경우에 비효율적이게 다시 본인을 복사하는 것을 방지하기 위해 `r = r`인 경우에는 그냥 본인을 반환하도록 예외 처리하였다. 그리고 기존 queue에 데이터가 있을 경우 다른 버그가 발생할수 있기에 대입되는 queue의 데이터를 초기화해주었다.

두번째 operator+=는 단순히 val값을 queue에 넣어주는 역할이기 때문에 `p + 3` 이런식으로 코드를 작성하면 3을 p 라는 queue에 push해주도록 했다.

```
using namespace std;

int main() {
    yhw::queue<int> q;
    yhw::queue<int> p;
    yhw::queue<int> r;
    q.push(1);
    q.push(2);
    q.push(3);

    p.push(4);
    p.push(5);
    p.push(6);

    r = q + p;
    r + 3;
    r + 6;
    r + 73;

    while (!r.empty()) {
        cout << r.front() << endl;
        r.pop();
    }
}
```

1
2
3
4
5
6
3
6
73
C:\ROBIT
이 창을

실행 결과를 보면 $q(1, 2, 3)$ 과 $p(4, 5, 6)$ 이 이어붙여진 것을 확인할 수 있고, $r+3, r+6, r+73$ 을 했을 때 정상적으로 r 에 `push()`된것을 확인할 수 있다.